



苏州大学
SOOCHOW UNIVERSITY



数据结构

任课教师：兰志强

邮 箱：zqlan@suda.edu.cn

地 址：未来科创中心703-3



兰志强

工学博士，讲师

- ✓ 2012~2016，集成电路设计与集成系统
- ✓ 2016~2018，电子科学与技术
- ✓ 2018~2022，信息与通信工程

办公地址：未来校区未来科创中心703-3

联系方式：zqlan@suda.edu.cn

■ 研究方向

- 智能感知物联网
- 多模态（声、光等）目标识别及定位



数据结构——20250...

群号: 1061923973

课程QQ群



问题：上课地点为？

答案：创新中心203

- 有问题随时说，我可以多解释
- 有意见随时提，大家一起进步
- 有事情要请假，有问题好商量
- 多思考练习，无他，唯手熟尔



202509数据科学-数...

扫一扫二维码打开或分享给好友

课程意见匿名反馈链接





目录

编号	章节	理论课时	实验课时
1	绪论	4	2
2	线性表	8	4
3	栈和队列	8	4
4	串	4	0
5	递归	6	2
6	数组和广义表	4	4
7	树和二叉树	8	4
8	图	8	4
9	查找	8	4
10	内排序	6	2
11	外排序	2	0
12	面向对象方法描述算法	4	4
13	复习	2	2
小计		72	36

➤ **考核占比（预计）：平时30% + 期中30% + 期末40%**



参考资料

- 数据结构教程，第6版·微课视频·题库版，李春葆；
- 数据结构(c语言版)，严蔚敏；
- 浙江大学，陈越<bilibili名称：“真的陈越姥姥”>，数据结构网课 (<https://github.com/smysophia/Data-Structure-Notes>)



- 什么是数据?
- 数据在计算机如何存储?
- 为什么需要数据结构?
- 有哪些数据结构?



大数据：如何组织？如何处理？



案例：学生管理系统

需要快速查找/修改学生个人信息（姓名、学号、身份证号、电话号码等）

- 如何实现？
- 如何查找？
- 具体如何存储？

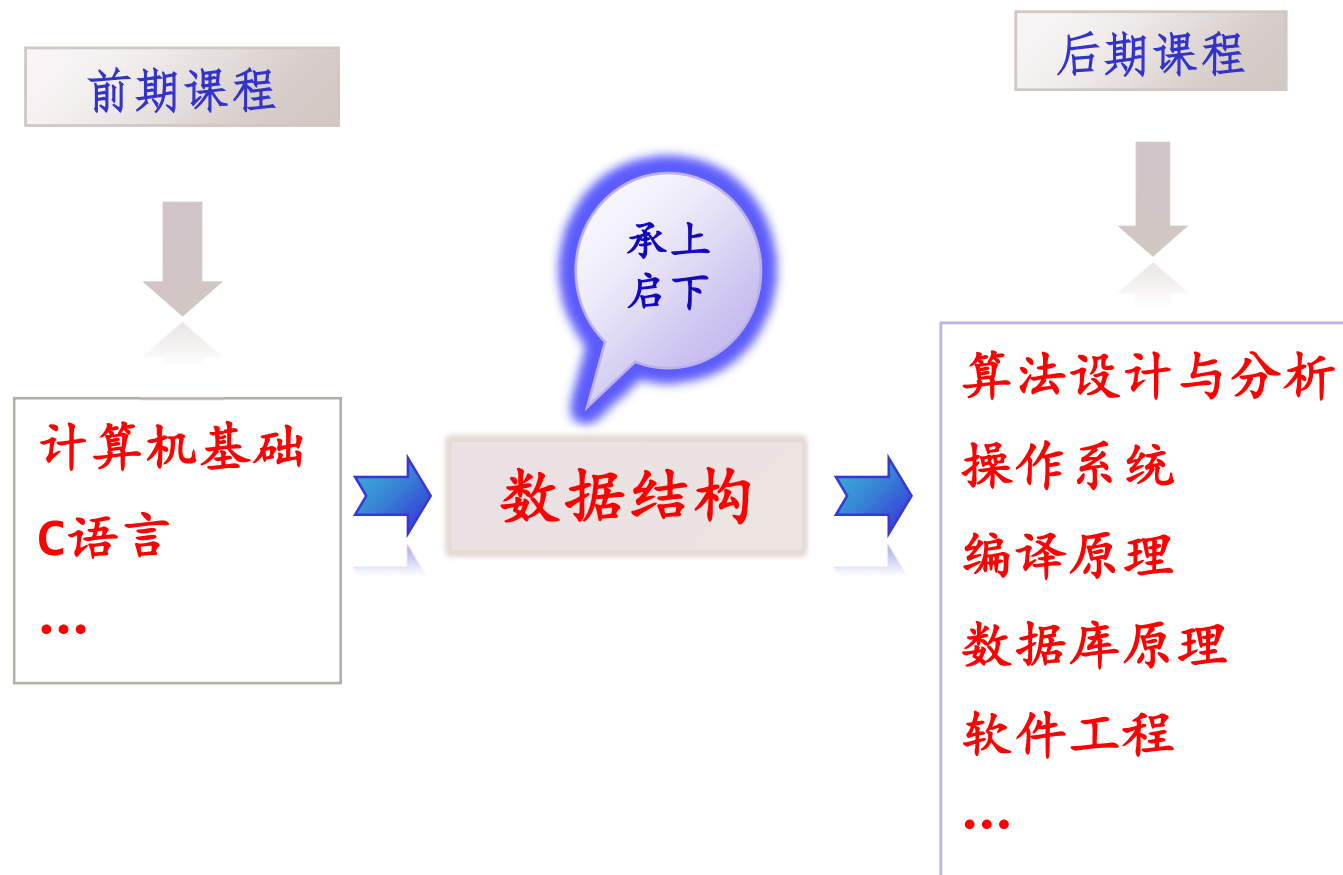


- 各种数据的逻辑结构描述。
- 各种数据的存储结构表示。
- 各种数据结构的运算定义。
- 设计实现运算的算法。
- 分析算法的效率。

基本数据组织和
数据处理方法

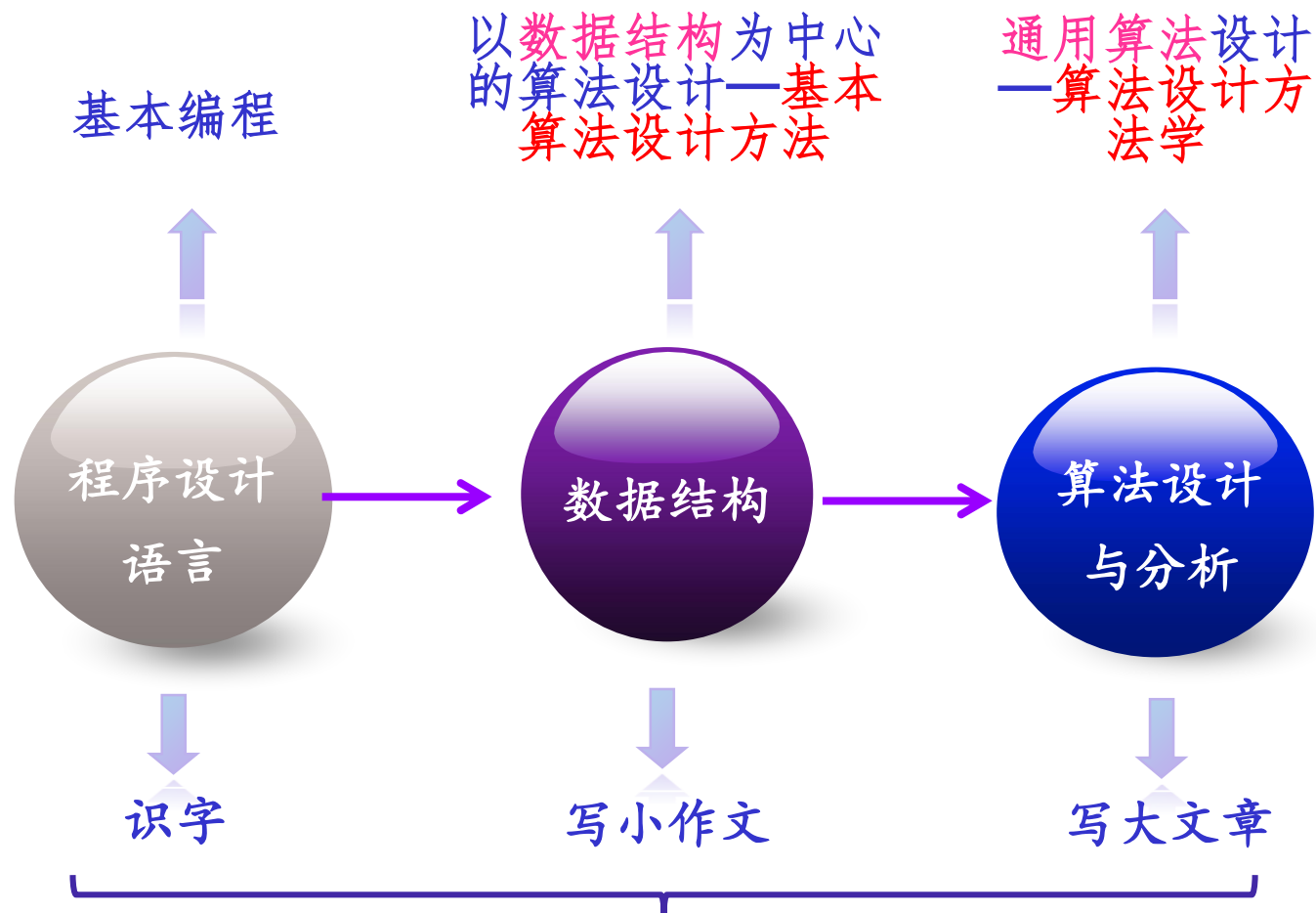


“数据结构”在计算机课程体系（偏软）中的地位





“数据结构”与程序设计类课程的关系

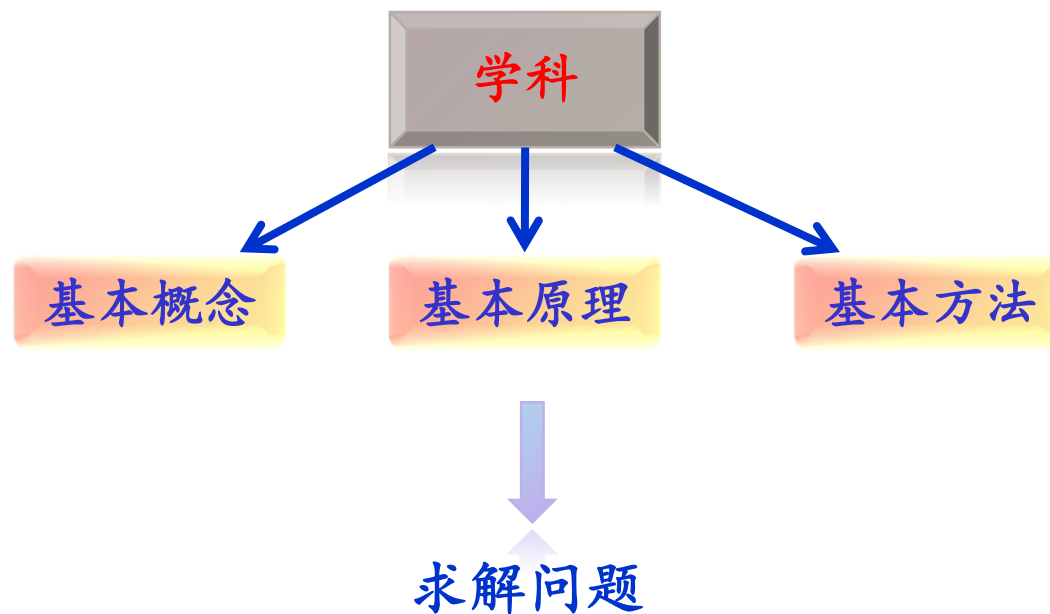


与语文学习过程类比



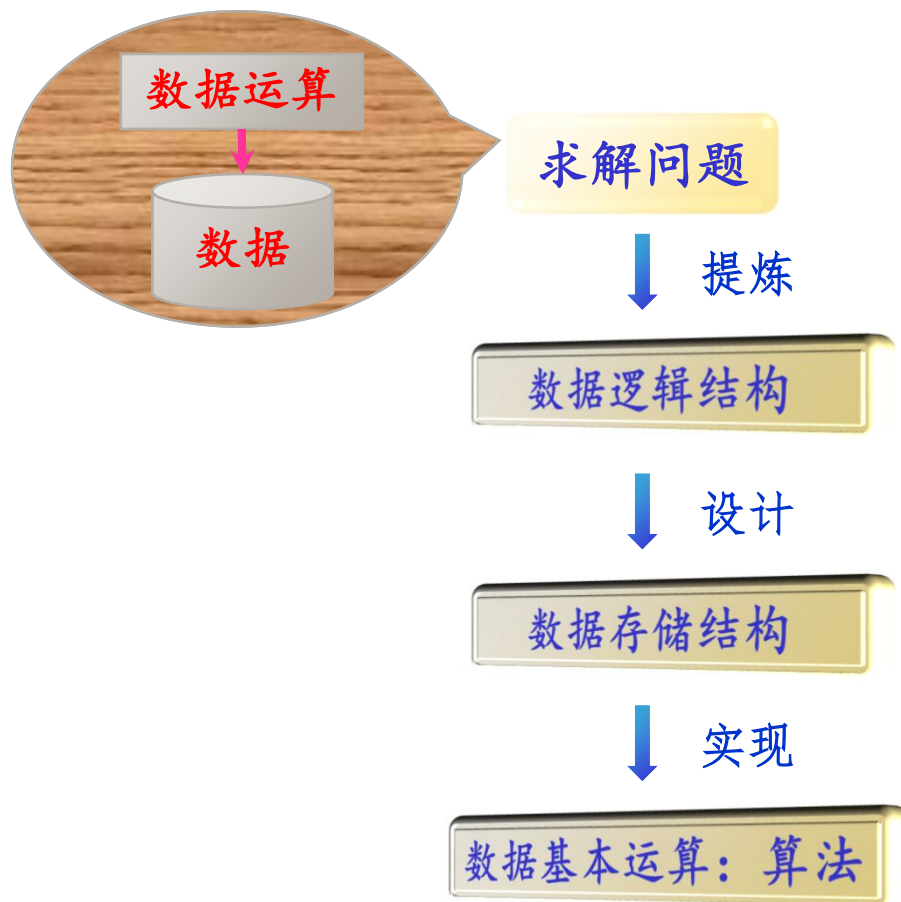
“数据结构”的学习目标

- ① 掌握数据结构的基本概念、基本原理和基本方法。





② 掌握数据的逻辑结构、存储结构及基本运算的实现过程。





- ③ 掌握算法基本的时间复杂度与空间复杂度的分析方法，能够设计出求解问题的**高效**算法。

关键：如何用数学方式抽象客观问题，有编程语言描述逻辑过程



同一求解问题通常有多种实现算法，通过时间复杂度与空间复杂度的分析，找出最好的实现算法。

例如，求 $1 + 2 + \dots n$ 。

算法1:

```
int fun1(int n)
{ int i, s=0;
  for (i=1; i<=n; i++)
    s+=i;
  return s;
}
```

算法2:

```
int fun2(int n)
{
  return (n+1)*n/2;
}
```

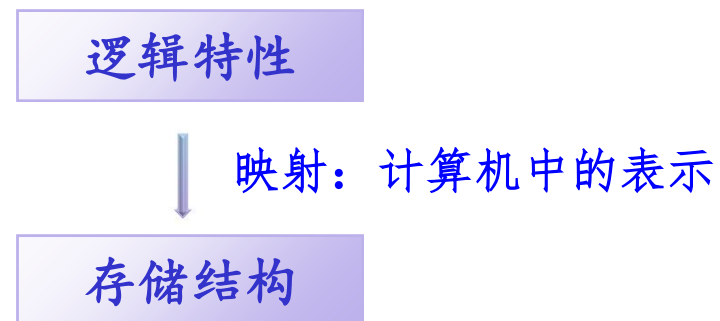
算法分析

算法2好于算法1



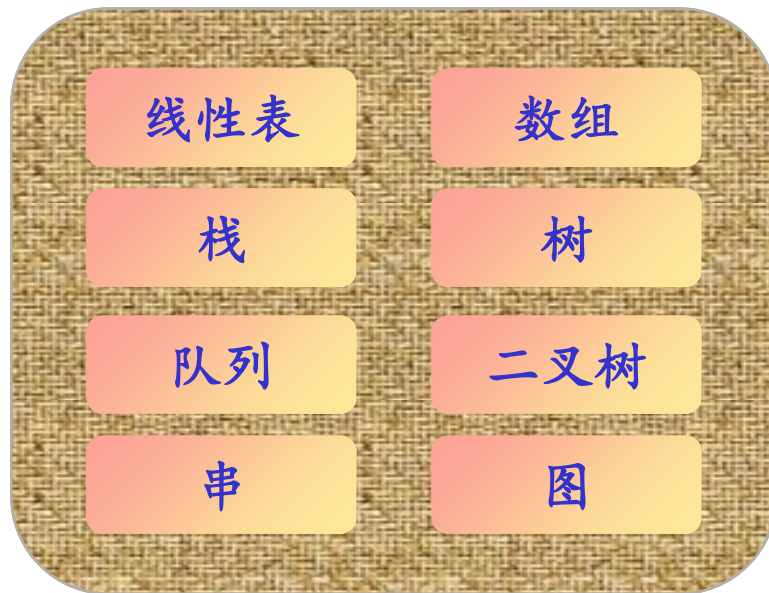
“数据结构”的学习方法

- ① 理解各种数据结构的逻辑特性和存储结构设计。





② 掌握各种数据结构算法设计的基本方法。



逻辑特性

映射：计算机中的表示

存储结构

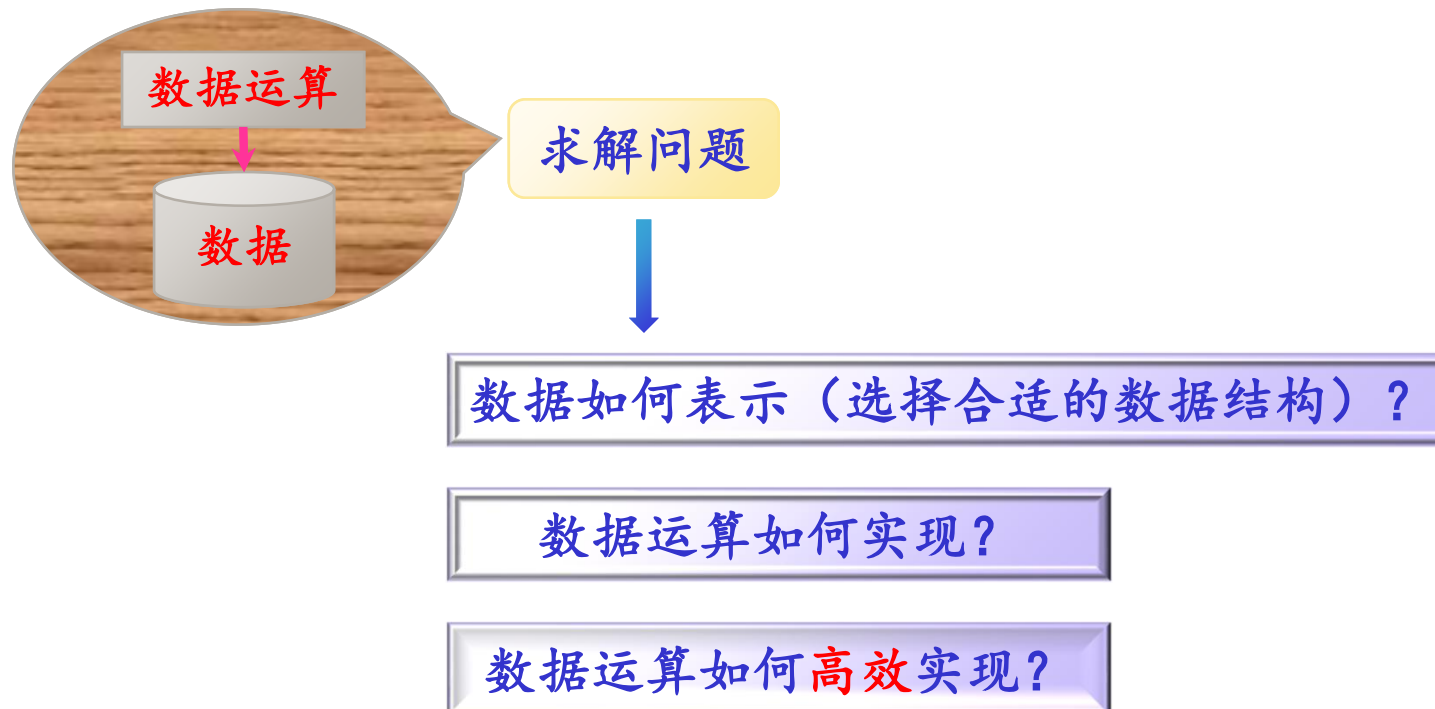
运算实现

算法设计

只有掌握了数据的存储结构表示，才能在此之上设计算法。



③ 利用各种数据结构来求解实际问题。





④ 演绎和归纳相结合。



培根：方法是旧的，问题是新的



苏州大学
SOOCHOW UNIVERSITY



第1章 绪论

提纲

CONTENTS

1.1 什么是数据结构

1.2 算法及其描述

1.3 算法分析

1.4 数据结构+算法=程序



1.1 什么是数据结构

1.1.1 数据结构的定义

数据结构中的几个概念



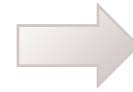
数据：所有能够输入到计算机中，且能被计算机处理的符号的集合。



Word文档



图像文档



都是数据

而数据结构中主要讨论结构化数据。



结构化数据示例

一个学生表

学号	姓名	性别	班号
1	张斌	男	9901
8	刘丽	女	9902
34	李英	女	9901
20	陈华	男	9902
12	王奇	男	9901
26	董强	男	9902
5	王萍	女	9901

数据项 (用于
描述数据元素)

数据
元素

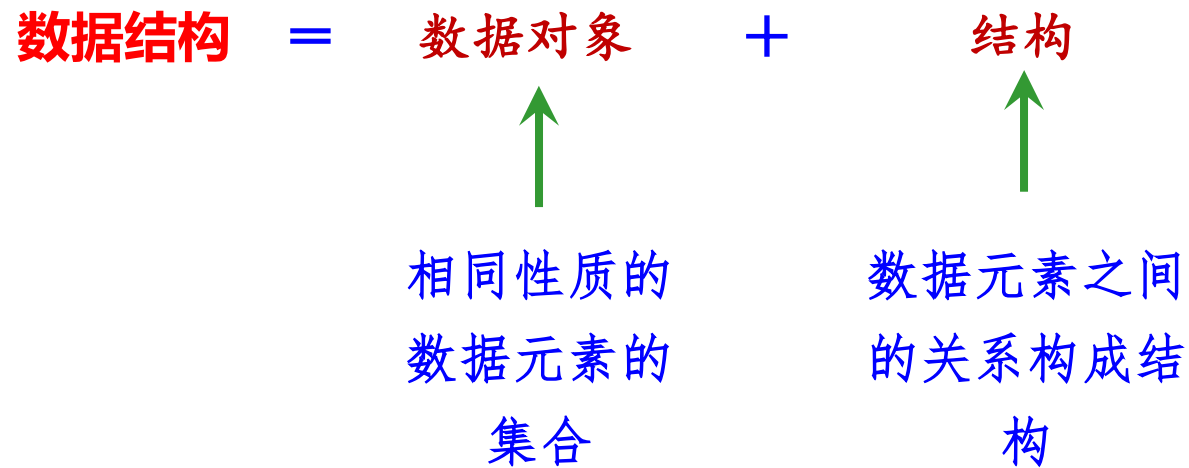


- **数据元素：**是数据（集合）中的一个“个体”，它是数据的基本单位。
- **数据项：**数据项是用来描述数据元素的，它是数据的最小单位。
- **数据对象：**具有相同性质的若干个数据元素的集合，如整数数据对象是所有整数的集合。

默认情况下，数据结构中讨论的数据都是**数据对象**。



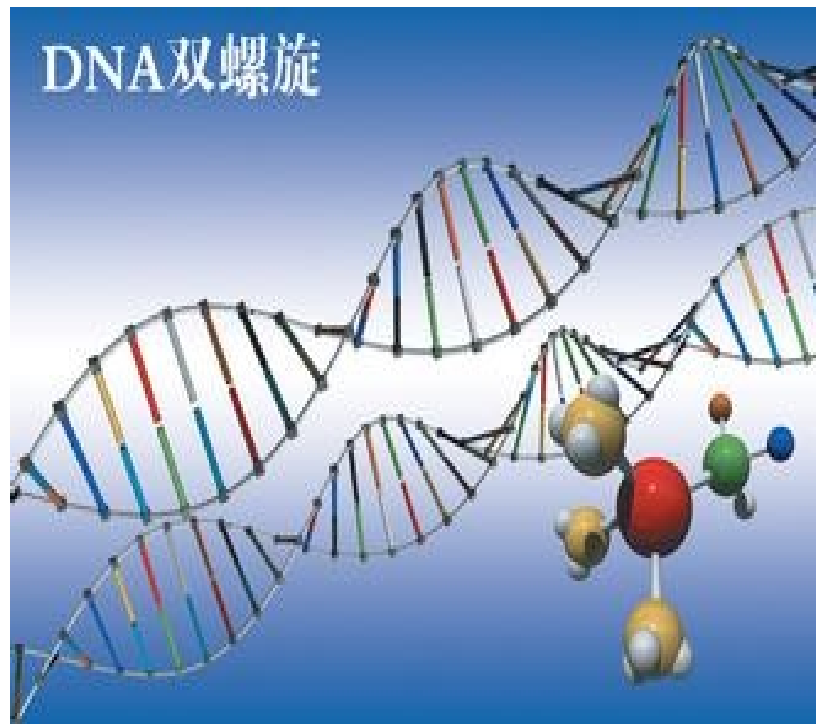
数据结构：是指带结构的数据元素的集合。





数据元素之间的关系 $\Rightarrow$ 结构, 现实世界的结构是纷繁复杂的

① 微观世界—DNA结构





② 宏观世界一建筑物的结构





数据结构中讨论的元素关系主要是指**相邻关系**或**邻接关系**。

学号	姓名	性别	班号	
1	张斌	男	9901	相邻
8	刘丽	女	9902	
34	李英	女	9901	不相邻
20	陈华	男	9902	
12	王奇	男	9901	
26	董强	男	9902	
5	王萍	女	9901	



一个数据结构的构成



- 数据元素之间的逻辑关系 $\Rightarrow$ 数据的**逻辑结构**。
- 数据元素及其关系在计算机存储器中的存储方式 $\Rightarrow$ 数据的**存储结构**（或**物理结构**）。
- 施加在该数据上的操作 $\Rightarrow$ **数据运算**。



1、数据的逻辑结构表示

数据的逻辑结构是面向用户的，它有多种表示形式。

① 学生表的逻辑结构表示1-表格

学号	姓名	性别	班号
1	张斌	男	9901
8	刘丽	女	9902
34	李英	女	9901
20	陈华	男	9902
12	王奇	男	9901
26	董强	男	9902
5	王萍	女	9901

直接来源于现实世界



② 学生表的逻辑结构表示2-二元组

二元组是一种通用的逻辑结构表示方法

一个二元组表示为：

$$B=(D, R)$$

其中，**B**是一种数据结构，它由数据元素的集合**D**和**D**上二元关系的集合**R**所组成。其中：

$D=\{ d_i \mid 1 \leq i \leq n, n \geq 0 \}$: 数据元素的集合

$R=\{ r_j \mid 1 \leq j \leq m, m \geq 0 \}$: 关系的集合



每个关系的用若干个序偶来表示：

- 序偶 $\langle x, y \rangle$ ($x, y \in D$) $\Rightarrow$ x 为第一元素， y 为第二元素。
- x 为 y 的（直接）前驱元素。
- y 为 x 的（直接）后继元素。
- 若某个元素没有前驱元素，则称该元素为开始元素；若某个元素没有后继元素，则称该元素为终端元素。

序偶 $\langle x, y \rangle$ 表示 x 、 y 是有向的，序偶 (x, y) 表示 x 、 y 是无向的



学号	姓名	性别	班号
1	张斌	男	9901
8	刘丽	女	9902
34	李英	女	9901
20	陈华	男	9902
12	王奇	男	9901
26	董强	男	9902
5	王萍	女	9901

每个学生记录用学号标识

二元组逻辑表示:

$\langle 1, 8 \rangle, \langle 8, 34 \rangle, \langle 34, 20 \rangle, \langle 20, 12 \rangle, \langle 12, 26 \rangle, \langle 26, 5 \rangle$



例如，如下数据为一个矩阵：

$$\begin{bmatrix} 2 & 6 & 3 & 1 \\ 8 & 12 & 7 & 4 \\ 5 & 10 & 9 & 11 \end{bmatrix}$$

对应的二元组表示为 $B=(D, R)$ ，其中：

$$D=\{2, 6, 3, 1, 8, 12, 7, 4, 5, 10, 9, 11\}$$

$$R=\{r1, r2\} \quad \text{其中, } r1 \text{ 表示行关系, } r2 \text{ 表示列关系}$$

$$r1=\{\langle 2,6\rangle, \langle 6,3\rangle, \langle 3,1\rangle, \langle 8,12\rangle, \langle 12,7\rangle, \langle 7,4\rangle, \\ \langle 5,10\rangle, \langle 10,9\rangle, \langle 9,11\rangle\}$$

$$r2=\{\langle 2,8\rangle, \langle 8,5\rangle, \langle 6,12\rangle, \langle 12,10\rangle, \langle 3,7\rangle, \langle 7,9\rangle, \\ \langle 1,4\rangle, \langle 4,11\rangle\}$$



③ 学生表的逻辑结构表示3-图形

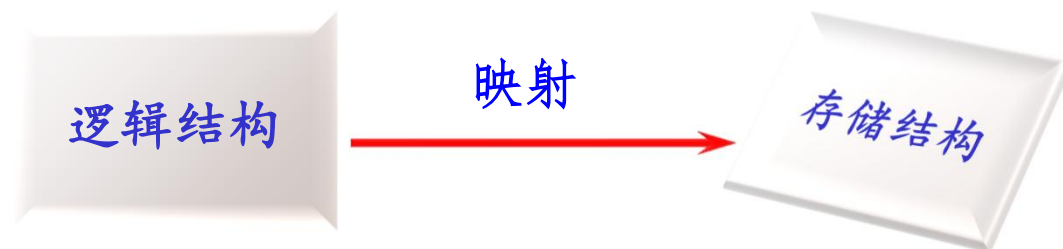
在学生表中，用学号标识每个学生记录，其逻辑结构用图形表示如下：





2、数据的存储结构表示

数据在计算机存储器中的存储方式就是存储结构。它是面向程序员的。



设计存储结构的这种映射应满足两个要求：

- 存储所有元素
- 存储数据元素间的关系



① 学生表存储结构1 - 结构体数组

存放学生表的结构体数组**Stud**定义如下：

```
struct
{
    int no;           //存储学号
    char name[8];     //存储姓名
    char sex[2];      //存储性别
    char class[4];    //存储班号
} Stud[7]={ {1,“张斌” ,“男” ,“9901”},
            ...,
            {5,"王萍","女","9901"} } ;
```



映射过程:

学生表的逻辑结构

学号	姓名	性别	班号
1	张斌	男	9901
8	刘丽	女	9902
34	李英	女	9901
20	陈华	男	9902
12	王奇	男	9901
26	董强	男	9902
5	王萍	女	9901



Stud数组起始地址

存储结构建立完毕



这种存储结构的特点:

- 所有元素占用一整块内存空间。
- 逻辑上相邻的元素，物理上也相邻。





② 学生表存储结构2—链表

存放学生表的链表的结点类型**StudType**声明如下：

```
typedef struct studnode
{   int no;                //存储学号
    char name[8];          //存储姓名
    char sex[2];           //存储性别
    char class[4];         //存储班号
    struct studnode *next; //存储指向下一个学生的指针
}   StudType;
```

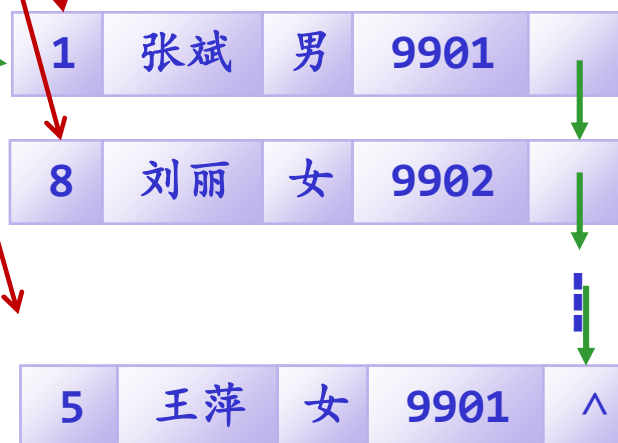



映射过程:

学生表的逻辑结构

学号	姓名	性别	班号
1	张斌	男	9901
8	刘丽	女	9902
34	李英	女	9901
20	陈华	男	9902
12	王奇	男	9901
26	董强	男	9902
5	王萍	女	9901

链表首
结点地
址head



存储结构建立完毕



这种存储结构的特点:

- 一个逻辑元素用一个结点存储，每个结点单独分配，所有结点的地址不一定是连续的。
- 用指针来表示逻辑关系。



链式存储结构



3、数据运算

数据运算是对数据的操作。分为两个层次：运算描述和运算实现。

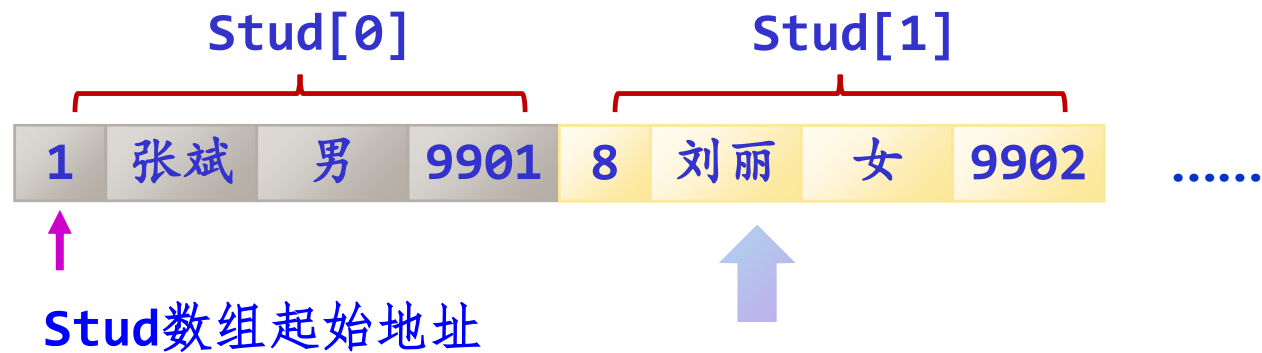
对于“学生表”这种数据结构，可以进行一系列的运算：

- 查找序号为2的学生姓名
- 增加一个学生记录；
- 删除一个学生记录；
- 查找性别为“女”的学生记录；
- 查找班号为“9902”的学生记录；
-

运算描述



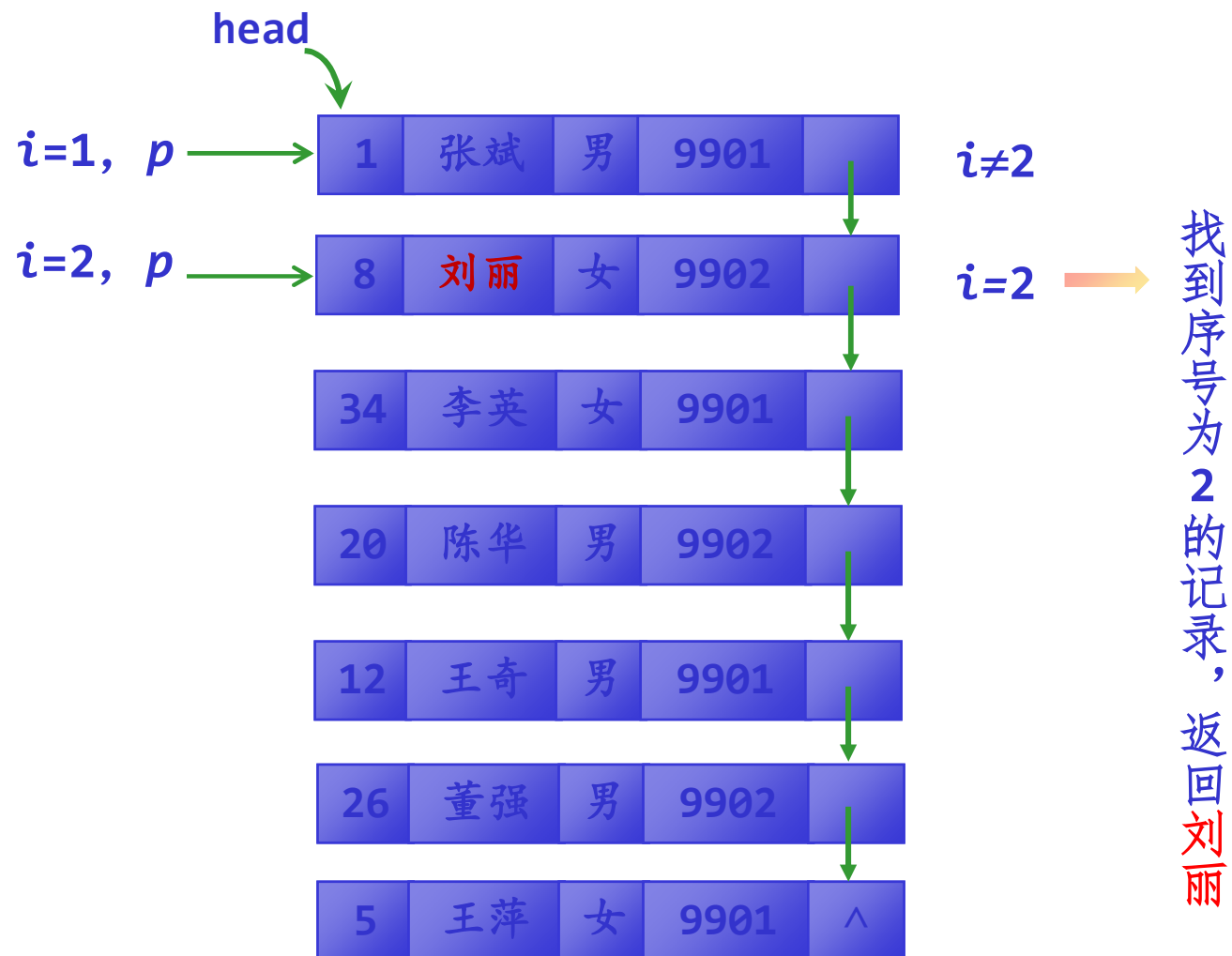
① 顺序存储结构中实现“查找序号为2的学生姓名”



直接找到**Stud[1]** 记录，返回**刘丽**



② 链式存储结构中实现 “查找序号为2的学生姓名”





结论

- 同一逻辑结构可以对应多种存储结构。
- 同样的运算，在不同的存储结构中，其实现过程是不同的。



思考题:

如何根据数据的逻辑结构设计相应的存储结构?



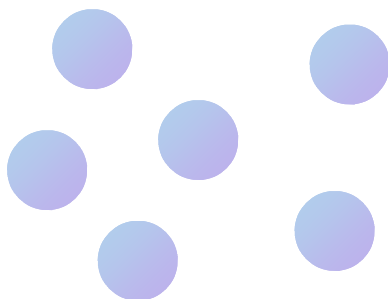
1.1.2 逻辑结构类型

各种各样的数据呈现出不同的逻辑结构，归纳为4种。

1、集合

元素之间关系：无。

特点：数据元素之间除了“属于同一个集合”的关系外，别无其他逻辑关系。是最松散的，不受任何制约的关系。



举例？



2、线性结构

元素之间关系：一对一。

特点：开始元素和终端元素都是唯一的，除此之外，其余元素都有且仅有一个前驱元素和一个后继元素。



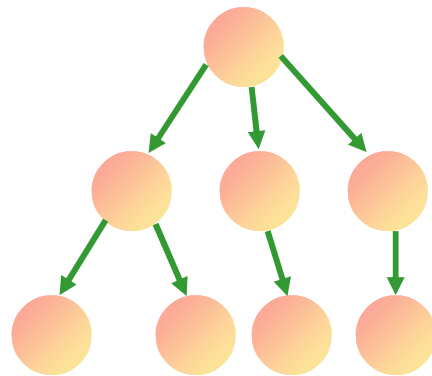
举例？



3、树形结构

元素之间关系：一对多。

特点：开始元素唯一，终端元素不唯一。除终端元素以外，每个元素有一个或多个后继元素；除开始元素外，每个元素有且仅有一个先驱元素。



举例？



【例1.3】有一种数据结构 $B2 = (D, R)$ ，其中

$D = \{48, 25, 64, 57, 82, 36, 75\}$

$R = \{r_1, r_2\}$

$r_1 = \{ \langle 25, 36 \rangle, \langle 36, 48 \rangle, \langle 48, 57 \rangle, \langle 57, 64 \rangle, \langle 64, 75 \rangle, \langle 75, 82 \rangle \}$

$r_2 = \{ \langle 48, 25 \rangle, \langle 48, 64 \rangle, \langle 64, 57 \rangle, \langle 64, 82 \rangle, \langle 25, 36 \rangle, \langle 82, 75 \rangle \}$

画出其逻辑结构表示，指出是什么类型？

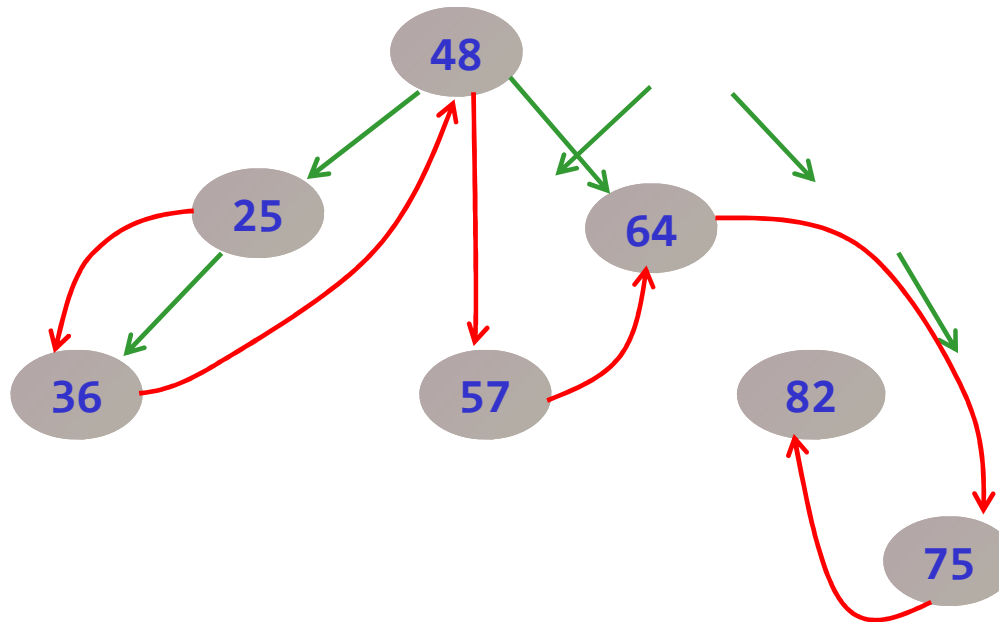


解

B2的逻辑结构图如下。

r_1 关系表示 $\Rightarrow$ r_1 为线性结构

r_2 关系表示 $\Rightarrow$ r_2 为树形结构

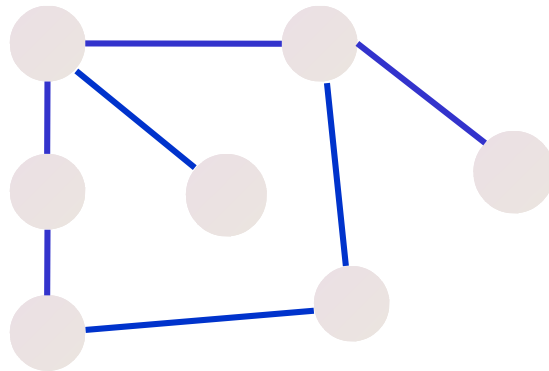




4、图形结构

元素之间关系：多对多。

特点：所有元素都可能有多多个前驱元素和多个后继元素。





1.1.3 存储结构类型

在软件开发中，人们设计了各种存储结构。归纳为4种基本的存储结构。

- 顺序存储结构
- 链式存储结构
- 索引存储结构
- 哈希（散列）存储结构

● 哈希（散列）存储结构

● 索引存储结构



1.1.4 数据类型和抽象数据类型

1、数据类型

在高级程序语言中提供了多种数据类型。不同数据类型的变量，其所能取的值的范围不同，所能进行的操作不同。

数据类型是一个值的集合和定义在此集合上的一组操作的总称。



例如，C/C++中的short int就是整型数据类型。

十、一、*、/ ... —— 一组操作



-32768~32767 —— 值的集合



例如，int数据类型：

int i=2,

j=5, k;

k=i+j;

...

int i=9999999999;

i**;

✓ 因为*i*、*j*和*k*都属于
int，而int提供了各种
运算，所以可以进行相
应运算。

✗

数据类型和数据结构的关系：数据类型就是已经实现了的数据结构。



2、抽象数据类型

抽象数据类型 (ADT) 指的是从求解问题的数学模型中抽象出来的数据逻辑结构和运算 (抽象运算), 而不考虑计算机的具体实现。



抽象数据类型 = 逻辑结构 + 抽象运算



例如，定义复数抽象数据类型Complex

一个复数的形式： $e_1 + e_2i$ 或 (e_1, e_2)

ADT Complex

{

数据对象:

$D = \{ e_1, e_2 \mid e_1, e_2 \text{ 均为实数} \}$

数据关系:

$R = \{ \langle e_1, e_2 \rangle \mid e_1 \text{ 是复数的实部, } e_2 \text{ 是复数的虚部} \}$



基本运算:

`AssignComplex(&z, v1, v2)`: 构造复数 z 。

`DestroyComplex(&z)`: 复数 z 被销毁。

`GetReal(z, &real)`: 返回复数 z 的实部值。

`GetImag(z, &Imag)`: 返回复数 z 的虚部值。

`Add(z1, z2, &sum)`: 返回两个复数 $z1$ 、 $z2$ 的和。

} **ADT Complex**

运算功能描述



Complex

ADT



编程实现该数据结构



抽象数据类型实质上就是对一个求解问题的形式化描述（与计算机无关），程序员可以在理解基础上实现它。



思考题

采用C/C++语言如何实现复数抽象数据类型Complex?



1.2 算法及其描述

1.2.1 什么是算法

数据元素之间的关系有逻辑关系和物理关系，对应的运算有基于逻辑结构的运算描述和基于存储结构的运算实现。

通常把基于存储结构的运算实现的步骤或过程称为算法。



更一般地，**算法**是“解决问题的处理步骤”。

- 数据是各种信息的表现形式
- 算法表现为“处理”和“数据”的结合



算法的五个重要的特性

(1) **有穷性**: 在有穷步之后结束, 算法能够停机。

(2) **确定性**: 无二义性。

(3) **可行性**: 可通过基本运算有限次执行来实现,
也就是算法中每一个动作能够被机械地执行。

(4) **输入**: 0个或多个输入

(5) **输出**: 1个或多个输出

} 表示存在数据处理



算法和程序的区别

不一定满足有穷性

- **程序**是指使用某种计算机语言对一个算法的具体实现，即具体要怎么做。
- **算法**侧重于对解决问题的方法描述，即要做什么。

一定满足有穷性

算法用计算机语言描述 $\Rightarrow$ 程序



【例】 考虑下列两段描述，这两段描述均不能满足算法的特性，试问它们违反了哪些特性？

(1) 描述一

```
void exam1()
{  int  n=2;
   while (n%2==0)
       n=n+2;
   printf("%d\n", n);
}
```

其中有一个死循环，违反了算法的有穷性特性。



(2) 描述二

```
void exam2()  
{  int x, y;  
    y=0;  
    x=5/y;  
    printf("%d, %d\n", x, y);  
}
```

其中包含除零错误，违反了
算法的可行性特性



1.2.2 算法描述



返回值 算法对应的函数名(形参列表)

{ //临时变量的定义

//实现由输入参数到输出参数的操作

...

}

函数体

- 返回值：通常为bool类型，表示算法是否成功执行。
- 形参列表：由输入型参数和输出型参数构成。

↕
算法输入

↕
算法输出



如何描述输出型参数?

C++语言中提供了一种引用运算符“&”用于描述输出型参数。

引用示例

```
int a=10;
```

```
int &b=a;
```

↑
引用

a
b 10

↑
两个变量共享内存空间

↓
a、*b*同步发生改变



示例：设计一个交换两个整数的算法。

编写一个函数swap1(x, y):

```
void swap1(int x, int y)
{
    int tmp;
    tmp=x; x=y; y=tmp;
}
```

} 交换形参x和y的值



当执行语句swap1(a, b)时, a和b实参值不会发生了交换。

分析：x、y既是输入型参数，也是输出型参数



改正方法1: 采用指针的方式来回传形参的值，需将上述函数改为：

```
void swap2(int *x, int *y)
{
    int tmp;

    tmp=*x;    //将x的值放在tmp中
    *x=*y;     //将x所指的值改为*y
    *y=tmp;    //将y所指的值改为tmp
}
```

} 交换形参x和y所指向的值

上述函数的调用改为swap2(&a, &b) ⇒ 比较复杂。



改正方法2: 采用引用型形参 $\Rightarrow$ 将输出型形参改为引用类型。

```
void swap(int &x, int &y)
//形参前的“&”符号不是指针运算符
{ int tmp=x;
  x=y; y=tmp;
}
```

} 交换形参x和y的值

当执行语句 `swap(a, b)` 时，形、实参的匹配相当于：

`int &x=a;` *//a为x的引用*

`int &y=b;` *//b为y的引用*

这样，*a*与*x*共享存储空间、*b*与*y*共享存储空间，因此执行函数后*a*和*b*的值发生了交换 $\Rightarrow$ 简单明了。



普通的参数传递

```
void fun1(int n)
{ int m=2;
  fun2(m);
  printf("%d\n", m);
}
```

实参



```
void fun2(int x)
{ x++;
  printf("%d\n", x);
}
```

普通形参

实参到形参单向值传递

fun1(m)



fun2(x)



引用类型的参数传递

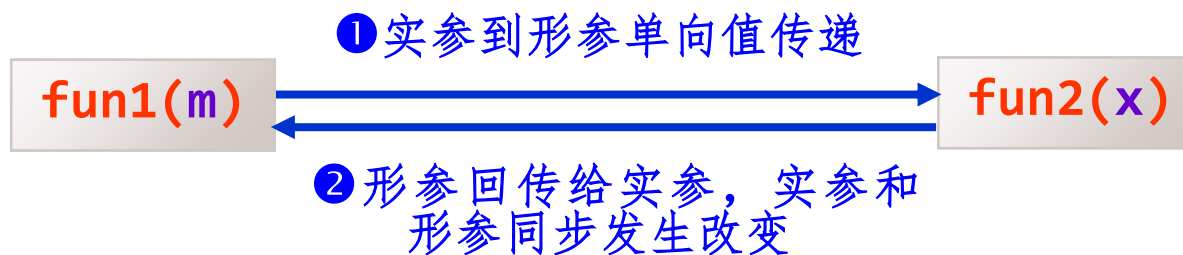
```
void fun1(int n)
{
    int m=2;
    fun2(m);
    printf("%d\n", m);
}
```

实参



```
void fun2(int &x)
{
    x++;
    printf("%d\n", x);
}
```

引用型形参





描述算法示例

【例1.5】 设计一个算法：求一元二次方程 $ax^2+bx+c=0$ 的根。

算法可以采用自然语言、流程图或者表格方式等来描述。

但是，一个学习计算机的学生应该使用某种计算机语言来描述算法。
本课程采用C/C++语言描述算法。

C++的作用是在描述算法时使用其提供的引用类型！



解

输入: a b c

solution

输出:

● 根个数
● x_1 x_2

```
int solution(float a, float b, float c,
            float &x1, float &x2)
{   float d, x1, x2;
    d=b*b-4*a*c;
    if (d>0)
    {   x1=(-b+sqrt(d))/(2*a);
        x2=(-b-sqrt(d))/(2*a);
        return 2;           //2个实根
    }
    else if (d==0)
    {   x1=(-b)/(2*a);
        return 1;           //1个实根
    }
    else                       //d<0的情况
        return 0;           //不存在实根
}
```



思考题

- ① 在用C/C++语言描述算法时，输入型参数和输出型参数如何设计？
- ② 一个算法只能用C/C++语言中的一个函数描述吗？



补充几个问题说明 (1/2)

- 算法描述方式：自然语言、伪码、计算机语言？

计算机专业学生应该具备熟练地采用计算机语言描述
算法的能力！

任正非辞退入职60天北大高材生，大佬们最不能容忍哪类员工

2017-09-07 18:04

一位入职华为仅60天的北大高材生曾写了个洋洋洒洒万言书，向老板任正非进言，历数华为的弊病和改进办法。而任正非的批复：“此人如果有精神病，建议送医院治疗；如果没病，建议辞退！”引发了网友热议。

不少人认为：该员工只是瞎操心，还不到辞退的地步。也有人认为，员工如此“天真”，怎么能怪任正非不给面子！

现今的华为，在任正非的领导下，飞速发展成世界知名的大公司，试想，它的战略方向岂是一个新入职员工能够规划得了？



补充几个问题说明 (2/2)

■ 采用什么语言描述算法：C、C++、C#、Java、Python?

均可。计算机专业学生最好采用C/C++!



📖 请问下面程序有什么错误?

➤ 有哪些错误或可改进的

➤ 为什么?

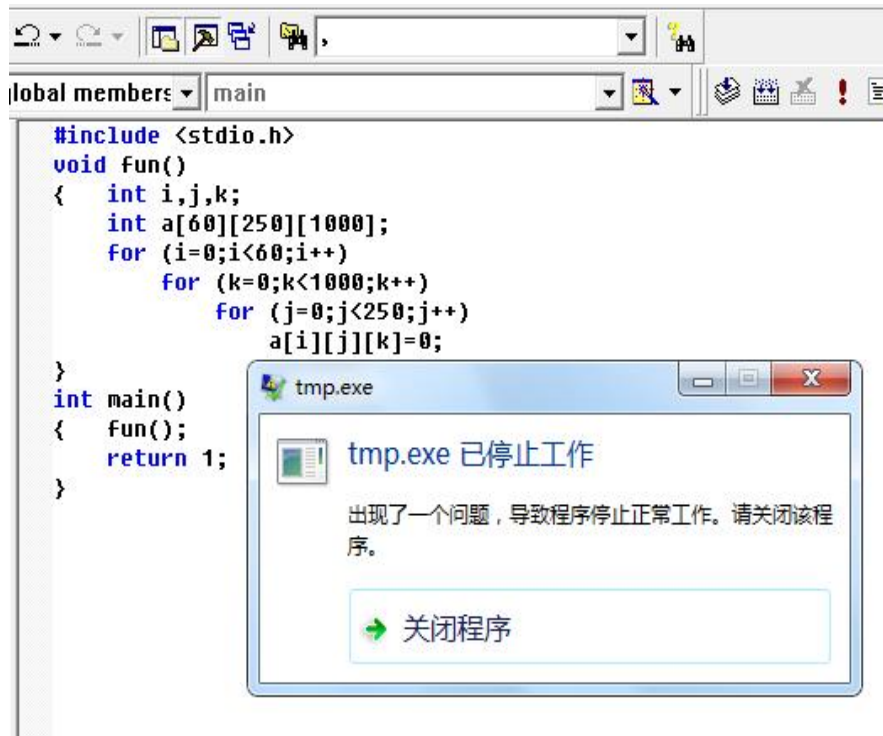
```
#include <stdio.h>

void fun()
{   int i,j,k;
int a[60][250][1000];
    for (i=0;i<60;i++)
        for (k=0;k<1000;k++)
            for (j=0;j<250;j++)
                a[i][j][k]=0;
}

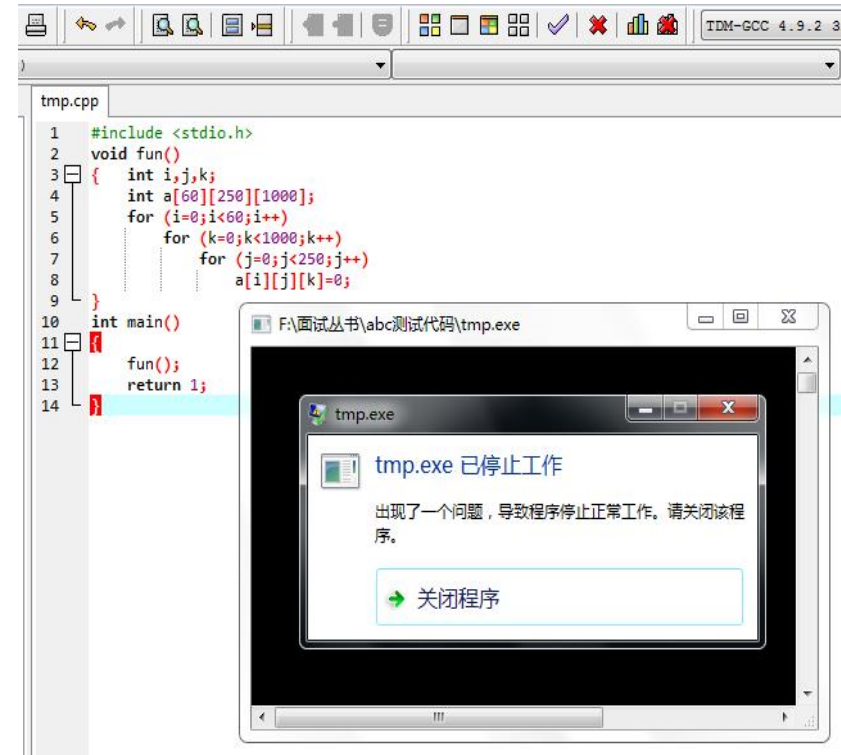
int main()
{   fun();
    return 1;
}
```




❏ 执行时栈溢出。



VC++ 6.0



Dev-C++ 5.11



出错原因:

- a数组在栈空间中分配
- 通常栈空间比较小，导致溢出

```
#include <stdio.h>

void fun()
{ int i,j,k;
  ← int a[60][250][1000];
    for (i=0;i<60;i++)
      for (k=0;k<1000;k++)
        for (j=0;j<250;j++)
          a[i][j][k]=0;
}

int main()
{ fun();
  return 1;
}
```

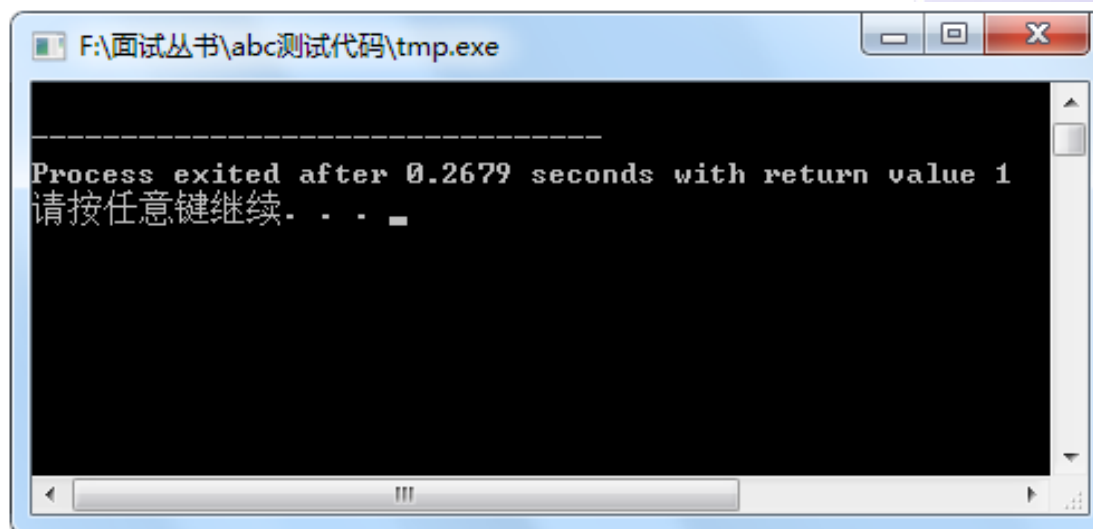


改正1:

在BSS中分配



```
#include <stdio.h>
int a[60][250][1000];
void fun()
{   int i,j,k;
    for (i=0;i<60;i++)
    for (k=0;k<1000;k++)
        for (j=0;j<250;j++)
            a[i][j][k]=0;
}
int main()
{   fun();
    return 1;
}
```





改正2:

执行时间减少为

$0.083/0.268=31\%$

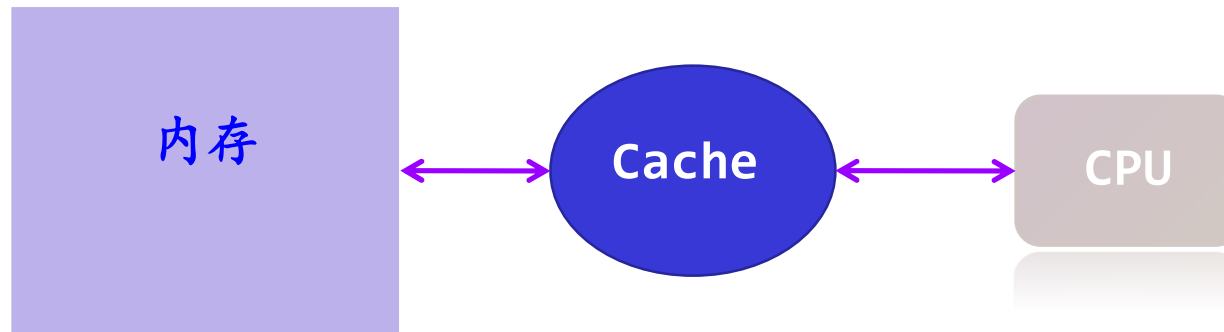


```
#include <stdio.h>
int a[60][250][1000];
void fun()
{ int i,j,k;
  for (i=0;i<60;i++)
    for (j=0;j<250;j++)
      for (k=0;k<1000;k++)
        a[i][j][k]=0;
}
int main()
{ fun();
  return 1;
}
```



原因:

数组按行序为主序排列



```
for (i=0;i<60;i++)  
  for (j=0;j<250;j++)  
    for (k=0;k<1000;k++)  
      a[i][j][k]=0;
```

按行序为主序操作数组元素

C/C++有助于培养学生的计算机系统观，为计算机组成、操作系统打下基础

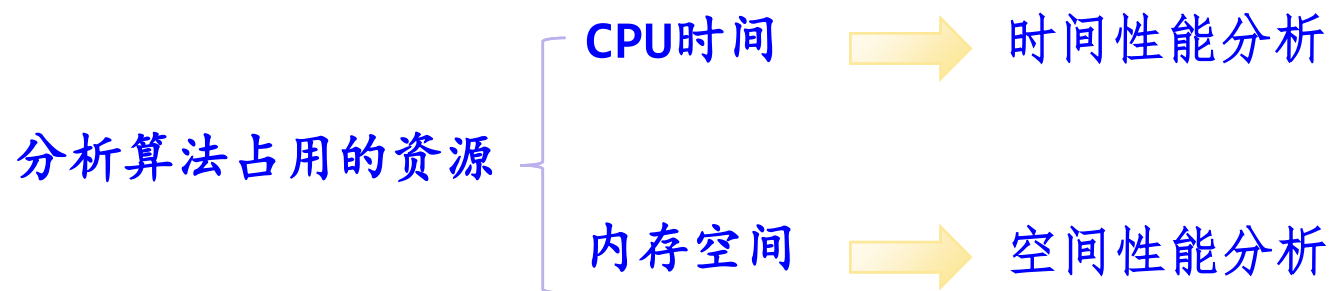


如何评估一个算法的好坏?



1.3 算法分析

1.3.1 算法分析概述



算法分析目的： 分析算法的时空效率以便改进算法性能。



示例

算法分析的目的是（ ）。

- A. 找出数据结构的合理性
- B. 研究算法中输入和输出的关系
- C. 分析算法的效率以求改进
- D. 分析算法的易读性和可行性

答：C。



示例

设计一个算法求 $1+(1+2)+(1+2+3)+\cdots+(1+2+3+\cdots+n)$, $n>2$ 。

```
int Sum1(int n)
{
    int sum=0;
    for (int i=1;i<=n;i++)
        for (int j=1;j<=i;j++) //求1+2+...+i
            sum+=j;
    return sum;
}
```

➡ 两重循环

```
int Sum2(int n)
{
    int sum=0,s=0;
    for (int i=1;i<=n;i++)
    {
        s+=i;
        sum+=s;
    }
    return sum;
}
```

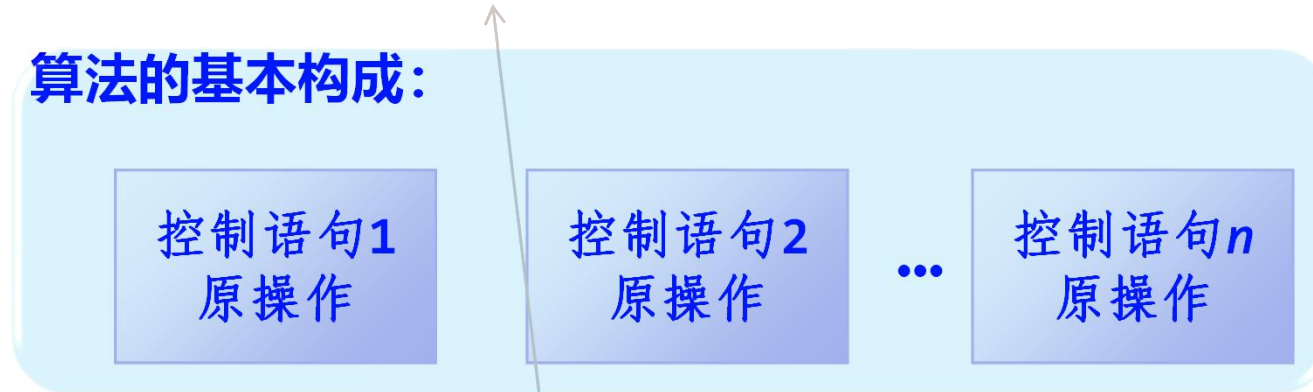
➡ 一重循环



1.3.2 算法时间复杂度分析

- 一个算法是由**控制结构**（顺序、分支和循环三种）和**原操作**构成的。

算法的基本构成：



- 顺序结构：按照所述顺序处理

- 分支结构：根据判断条件改变执行流程

- 循环结构：当条件成立时，反复执行给定的处理操作

指固有数据类型
的操作，如+、-、
*、/、++和--等



例如：

```
void fun(int a[], int n)
{
    int i;
    for (i=0;i<n;i++)
        a[i]=2*i;
    for (i=0;i<n;i++)
        printf("%d", a[i]);
    printf("\n");
}
```

原操作



算法分析方式

① 事后分析统计方法：编写算法对应程序，统计其执行时间。

- 编写程序的语言不同
- 执行程序的环境不同
- 其他因素

所以不能用绝对执行时间进行比较。

② 事前估算分析方法：撇开上述因素，认为算法的执行时间是问题规模 n 的函数。 ✓



① 分析算法的执行时间

- 求出算法所有原操作的执行次数（也称为**频度**），它是**问题规模 n** 的函数，用 **$T(n)$** 表示。
- 算法执行时间大致 = 原操作所需的时间 $\times T(n)$ 。所以 **$T(n)$** 与算法的执行时间**成正比**。为此用 **$T(n)$** 表示算法的执行时间。
- 比较不同算法的 **$T(n)$** 大小得出算法执行时间的好坏。

用于表示求解问题大小的正整数，如 **n** 个记录排序



【例1-6】求两个 n 阶方阵的相加 $C=A+B$ 的算法如下，分析其时间复杂度。

```
#define MAX 20          //定义最大的方阶
void matrixadd(int n, int A[MAX][MAX], int B[MAX][MAX],
               int C[MAX][MAX])
{   int i, j;
    for (i=0;i<n;i++)           //①
        for (j=0;j<n;j++)       //②
            C[i][j]=A[i][j]+B[i][j]; //③
}
```



```
#define MAX 20    //定义最大的方阶

void matrixadd(int n, int A[MAX][MAX],
               int B[MAX][MAX], int C[MAX][MAX])
{ int i, j;
  for (i=0;i<n;i++)
    for (j=0;j<n;j++)
      C[i][j]=A[i][j]+B[i][j]; //③
}
```

解：除变量定义语句外，该算法包括3个可执行语句①、②和③。

——①—— 频度为 $n+1$ ，循环体执行 n 次

——②—— 频度为 $n(n+1)$

——③—— 频度为 n^2



所有语句频度之和为：

$$\begin{aligned} T(n) &= n+1+n(n+1)+n^2 \\ &= 2n^2+2n+1 \end{aligned}$$

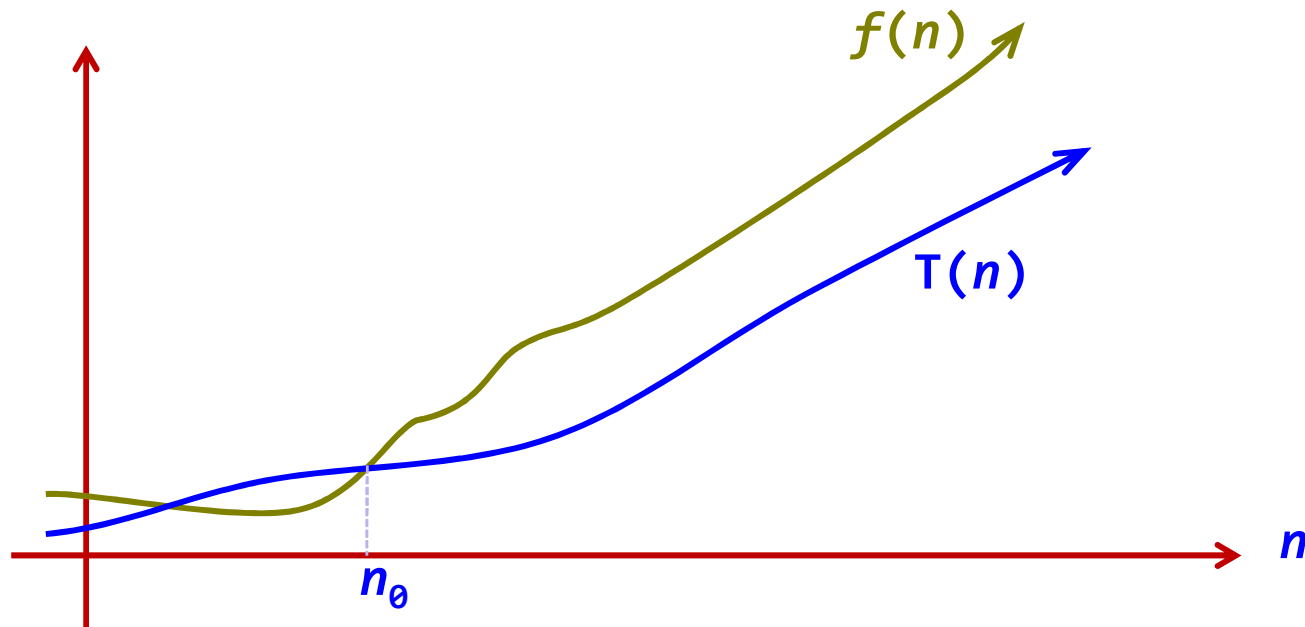


② 算法的执行时间用时间复杂度来表示

算法中执行时间 $T(n)$ 是问题规模 n 的某个函数 $f(n)$ ，记作：

$$T(n) = O(f(n))$$

记号“ O ”读作“**大O**”，它表示随问题规模 n 的增大算法执行时间的增长率和 $f(n)$ 的增长率相同。 $\Rightarrow$ **趋势分析**





“O”的形式定义为：

$T(n) = O(f(n))$ 表示存在一个正的常数 c ，使得当 $n \geq n_0$ 时都满足：

$$|T(n)| \leq c|f(n)|$$

$f(n)$ 是 $T(n)$ 的上界

这种上界可能很多，通常取最接近的上界，即紧凑上界

大致情况：

$$\lim_{n \rightarrow \infty} \frac{T(n)}{f(n)} = c$$



例如：

$$T(n)=2n^3-1000n+2 = O(n^3) \quad \lim_{n \rightarrow \infty} \frac{2n^3-1000n+2}{n^3} = 2$$

$$T(n)=2n^3-1000n+2 \neq O(n^2) \quad \lim_{n \rightarrow \infty} \frac{2n^3-1000n+2}{n^2} = \infty$$

$$T(n)=2n^3-1000n+2 \neq O(n^4) \quad \lim_{n \rightarrow \infty} \frac{2n^3-1000n+2}{n^4} = 0$$



本质上讲，是一种 $T(n)$
最高数量级的比较

也就是只求出 $T(n)$ 的最高阶，忽略其低阶项和常系数，这样既可简化 $T(n)$ 的计算，又能比较客观地反映出当 n 很大时算法的时间性能。

$$\text{例如: } T(n) = 2n^2 + 2n + 1 = O(n^2)$$



一般地：

- 一个没有循环的算法的执行时间与问题规模 n 无关，记作 $O(1)$ ，也称作常数阶。
- 一个只有一重循环的算法的执行时间与问题规模 n 的增长呈线性增大关系，记作 $O(n)$ ，也称线性阶。
- 其余常用的算法时间复杂度还有平方阶 $O(n^2)$ 、立方阶 $O(n^3)$ 、对数阶 $O(\log_2 n)$ 、指数阶 $O(2^n)$ 等。



各种不同算法时间复杂度的比较关系如下：

$$O(1) < O(\log_2 n) < O(n) < O(n \log_2 n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$$

多项式阶

指数阶

算法时间性能比较：假如求同一问题有两个算法： A 和 B ，如果算法 A 的平均时间复杂度为 $O(n)$ ，而算法 B 的平均时间复杂度为 $O(n^2)$ 。

一般情况下，认为算法 A 的时间性能好比算法 B 。



示例

某算法的时间复杂度为 $O(n^2)$ ，表明该算法的（ ）。

- A. 问题规模是 n^2
- B. 执行时间等于 n^2
- C. 执行时间与 n^2 成正比
- D. 问题规模与 n^2 成正比

答：C。



示例

下面几种算法时间复杂度中，时间复杂度最高的是（ ）。

A. $O(n \log_2 n)$ B. $O(n^2)$ C. $O(n)$ D. $O(2^n)$

答：D。



简化的算法时间复杂度分析

算法中的**基本操作**一般是最深层循环内的**原操作**。

算法执行时间大致 = **基本操作**所需的时间 $\times$ 其运算次数。



在算法分析时，计算 $T(n)$ 时仅仅考虑**基本操作**的运算次数。



【例1.6】求两个 n 阶方阵的相加 $C=A+B$ 的算法如下，分析其时间复杂度。

```
#define MAX    20    //定义最大的方阶
void matrixadd(int n, int A[MAX][MAX], int B[MAX][MAX],
               int C[MAX][MAX])
{   int i, j;
    for (i=0;i<n;i++)
        for (j=0;j<n;j++)
            C[i][j]=A[i][j]+B[i][j];
}
```

基本操作



```
void matrixadd(int n, int A[MAX][MAX], int B[MAX][MAX],
               int C[MAX][MAX])
{   int i, j;
    for (i=0;i<n;i++)
        for (j=0;j<n;j++)
            C[i][j]=A[i][j]+B[i][j];
}
```

解：该算法中的基本操作是两重循环中最深层的语句 $C[i][j]=A[i][j]+B[i][j]$ ，分析它的频度，即：

$$\begin{aligned} T(n) &= \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} 1 = \sum_{i=0}^{n-1} n = n \sum_{i=0}^{n-1} 1 = n * n = n^2 \\ &= O(n^2) \end{aligned}$$

这种简化的时间复杂度分析方法得到的结果相同，但分析过程更简单。



示例

下列程序段的时间复杂度是（ ）。

```
count=0;  
for(k=1;k<=n;k*=2)  
    for(j=1;j<=n;j++)  
        count++;
```

基本操作

A. $O(\log_2 n)$

B. $O(n)$

C. $O(n \log_2 n)$

D. $O(n^2)$

说明：本题为2014年全国考研题



【例1-7】分析以下算法的时间复杂度。

```
void func(int n)
{  int i=0, s=0;
   while (s<n)
   {  i++;
      s=s+i;
   }
}
```

} 基本操作



```
void func(int n)
{  int i=0, s=0;
  while (s<n)
  {  i++;
     s=s+i;
  }
}
```

解：对于while循环语句，设执行的次数为 m ，
变量 i 从0开始递增1，直到 m 为止，有：

循环结束： $s=m(m+1)/2 \geq n$ ，或者 $m(m+1)/2+k=n$ 。

↑
用于修正的常量

则：

$$m = \frac{-1 + \sqrt{8n + 1 - 8k}}{2}$$

$$T(n) = m = O(\sqrt{n})$$



最好、最坏和平均时间复杂度分析

定义： 设一个算法的输入规模为 n ， D_n 是所有输入的集合，任一输入 $I \in D_n$ ， $P(I)$ 是 I 出现的概率，有 $\sum_{I \in D_n} P(I) = 1$ ， $T(I)$ 是算法在输入 I 下的执行时间，则算法的**平均时间复杂度**为：

$$E(n) = \sum_{I \in D_n} P(I) \times T(I)$$



例如，10个1~10的整数序列递增排序 ($n=10$) :

$$I_1 = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$$

$$I_2 = \{2, 1, 3, 4, 5, 6, 7, 8, 9, 10\}$$

...

$$I_m = \{10, 9, 8, 7, 6, 5, 4, 3, 2, 1\}$$

构成 D_n

↑
所有可能的初始序列有 m 个, $m=10!$ $\Rightarrow P(I)=1/m$



算法的**最坏时间复杂度**为: $W(n)=\text{MAX}\{T(I)\}$
 $I \in D_n$

算法的**最好时间复杂度**为: $B(n)=\text{MIN}\{T(I)\}$
 $I \in D_n$

一种或几种特殊情况



【例1.8】 以下算法用于求含 n 个整数元素的序列中前 i ($1 \leq i \leq n$) 个元素的最大值。分析该算法的最好、最坏和平均时间复杂度。

```
int fun(int a[], int n, int i)
{
    int j, max=a[0];
    for (j=1;j<=i-1;j++)
        if (a[j]>max) max=a[j];
    return(max);
}
```



```
int fun(int a[], int n, int i)
{
    int j, max=a[0];
    for (j=1;j<=i-1;j++)
        if (a[j]>max) max=a[j];
    return(max);
}
```

解：算法的主要时间花费在元素比较上：

- i 的取值范围为 $1 \sim n$ ，共 n 种情况。
- 在等概率情况（每种情况的概率为 $1/n$ ）。
- 求前 i 个元素的最大值时， $\text{max}=\text{a}[0]$ ，将其与 $\text{a}[1..i-1]$ 元素进行比较。比较次数 $= (i-1) - 1 + 1 = i-1$ 次。

$$T(n) = \sum_i^n \frac{1}{n} \times (i-1) = \frac{1}{n} \sum_i^n (i-1) = \frac{n-1}{2}$$

$= O(n)$ ← 平均时间复杂度



```
int fun(int a[], int n, int i)
{
    int j, max=a[0];
    for (j=1;j<=i-1;j++)
        if (a[j]>max) max=a[j];
    return(max);
}
```

- $i=1$ 时, 比较次数为0 $\Rightarrow$ 算法的最好复杂度 $B(n)=O(1)$ 。
- $i=n$ 时, 比较次数为 $n-1$ $\Rightarrow$ 算法的最坏复杂度 $W(n)=O(n)$ 。



1.3.3 算法空间复杂度分析

空间复杂度：用于量度一个算法运行过程中临时占用的存储空间大小。

一般也作为问题规模 n 的函数，采用数量级形式描述，记作：

$$S(n) = O(g(n))$$

若一个算法的空间复杂度为 $O(1)$ ，则称此算法为原地工作或就地工作算法。



示例

算法的空间复杂度是指（ ）。

- A. 算法中输入数据所占用的存储空间的大小
- B. 算法本身所占用的存储空间的大小
- C. 算法中所占用的所有存储空间的大小
- D. 算法中需要的辅助变量所占用存储空间的大小

答：D。



示例

某算法的空间复杂度为 $O(1)$, 则 ()。

- A. 该算法执行不需要任何辅助空间
- B. 该算法执行所需辅助空间大小与问题规模 n 无关
- C. 该算法执行不需要任何空间
- D. 该算法执行所需空间大小与问题规模 n 无关

答: **B**。



为什么空间复杂度分析只考虑临时占用的存储空间?

```
int max(int a[], int n)
{
    int i, maxi=0;
    for (i=1; i<=n; i++)
        if (a[i]>a[maxi])
            maxi=i;
    return a[maxi];
}
```

如果max函数中再考虑形参a的空间，就重复累计了执行整个算法所需的空间。

max算法的空间复杂度为 $O(1)$

max()

max(b, n)

maxfun()

```
void maxfun()
{
    int b[]={1, 2, 3, 4, 5}, n=5;
    printf("Max=%d\n", max(b, n));
}
```

maxfun算法中为b数组分配了相应的内存空间，其空间复杂度为 $O(n)$



示例

分析如下算法的空间复杂度。

```
int fun(int n)
{  int i, j, k, s;
    s=0;
    for (i=0;i<=n;i++)
        for(j=0;j<=i;j++)
            for (k=0;k<=j;k++)
                s++;
    return(s);
}
```

临时占用的
存储空间：
函数体内分
配的空间

解：算法中临时分配的变量个数与问题规模 n 无关，所以空间复杂度均为 $O(1)$ 。



1.4 数据结构+算法=程序

1.4.1 程序和数据结构

- 程序总是以某些数据为处理对象。将松散、无组织的数据按某种要求组成一种数据结构，对于设计一个简明、高效、可靠的程序是大有益处的。
- 沃思：程序就是在数据的某些特定的表示方法和结构的基础上，对抽象算法的具体表述，所以说程序离不开数据结构。
- 程序是通过某种程序设计语言描述的，程序设计语言具有实现数据结构和算法的机制，其中类型声明与对象定义用于实现数据结构，而语句实现实现算法，描述程序的行为。



1.4.2 算法和程序

- 由程序设计语言描述的算法就是计算机程序。
- 对一个求解问题而言，算法就是解题的方法，没有算法，程序就成了无本之末，无源之水，有了算法，将它表示成程序是不困难的。
- 算法是程序的“灵魂”。算法在整个计算机科学中的地位都是极其重要的。



1.4.3 算法和数据结构



为什么分析算法时要将数据结构一并讨论？



数据结构角度求解问题的过程

ADT = 逻辑结构 + 抽象运算 (功能描述)

① 问题描述

映射

存储结构₁

...

存储结构_n

运算实现

② 设计存储结构

算法₁₁

...

算法_{1m}

算法_{n1}

...

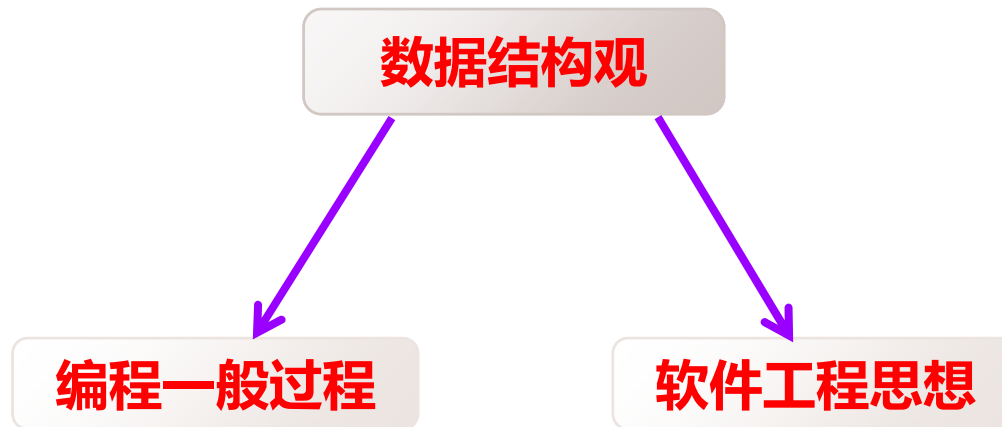
算法_{nm}

③ 算法设计

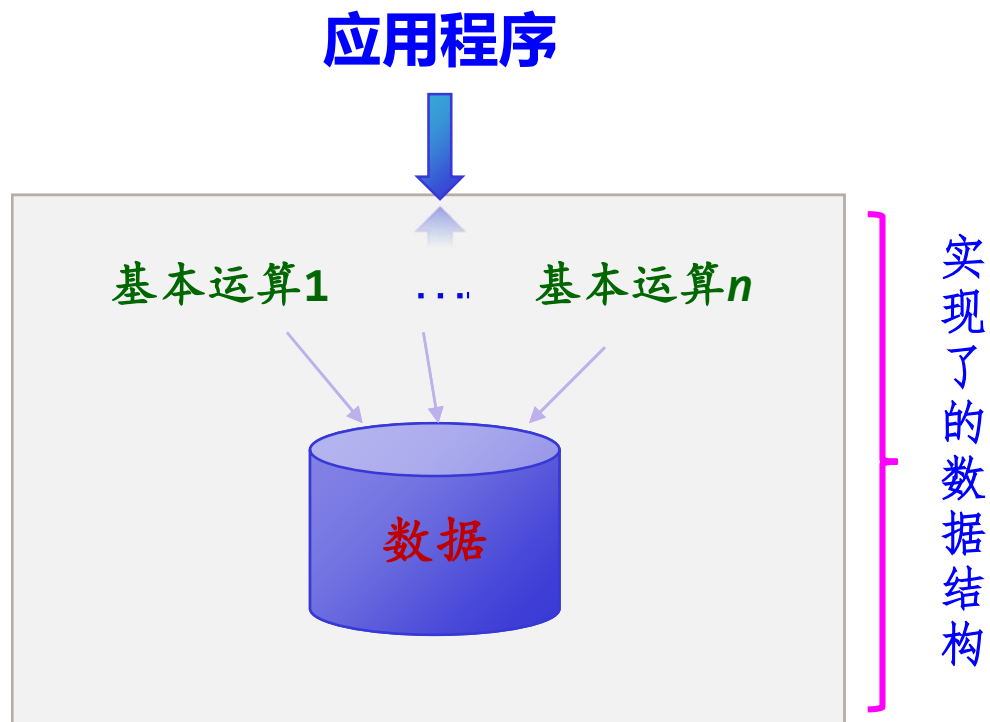
算法分析

最佳算法

④ 算法分析



- 问题求解 $\Rightarrow$ 算法设计
- 算法实现 $\Rightarrow$ 选择合适的存储结构
- 存储结构设计取决于数据的逻辑结构，并且为运算高效实现服务。



- 程序员可以直接使用它来存放数据——作为存放数据的容器。
- 程序员可以直接使用它的基本运算——完成更复杂的功能。



1.4.4 数据结构的发展

与数据结构历史相关的计算机科学家

Donald Ervin Knuth (高德纳) 1938年1月10日出生



- **1974**年获得图灵奖，以及无数奖项
- 称为最伟大的计算机科学家之一
- **《计算机程序设计的艺术》**系列，开始于他念博士期间，计划出七卷，第一卷《基本算法》于**1968**年出版，第二卷《半数字化算法》于**1969**年出版，第三卷《排序与搜索》于**1973**年出版，第四卷《组合算法》于**2008**年出版。《计算机程序设计的艺术》一书以其内容的丰富和深刻喻为经典，有人甚至称之为“计算机的圣经”



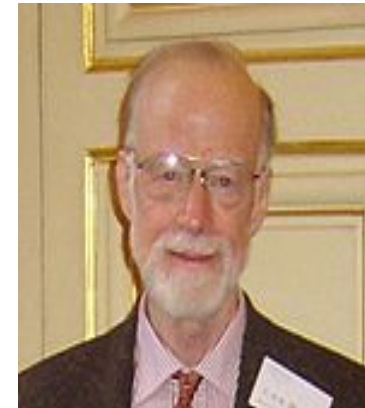
Niklaus Wirth, 1934年2月15日出生于瑞士



- **Niklaus Wirth**是著名的**Pascal**语言设计者 之一。
- 凡是学过一点计算机知识的人大概都知道“数据结构+算法= 程序”这一著名公式。提出这一公式并以此作为其一本专著的书名，并提出结构化程序设计这一革命性概念
- 沃思在其他方面也有许多创造，为了定义和描述语言，沃思对著名的“巴科斯-诺尔范式”**BNF**进行了扩充，成为**EBNF**（**Extended BNF**）。
- **1984**年获得图灵奖



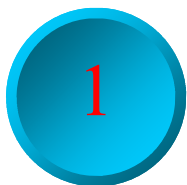
C.A.R.Hoare (1934年~)



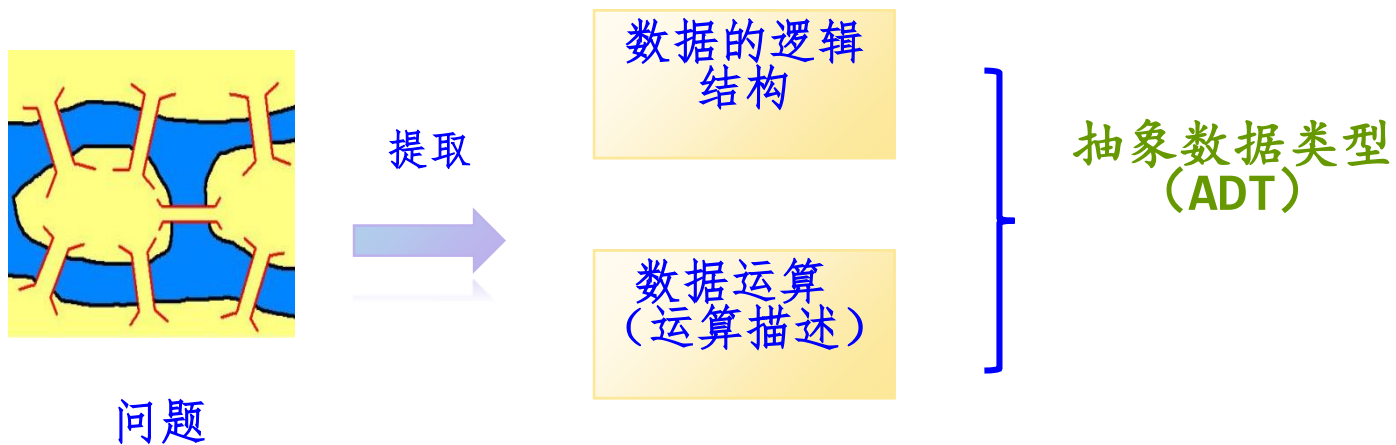
- 1960年发布了使他闻名于世的快速排序算法（Quick Sort），这个算法也是当前世界上使用最广泛的算法之一
- 领导了Algol 60第一个商用编译器的设计与开发
- 从1977年开始，Tony Hoare博士任职于牛津大学，投身于计算系统的精确性的研究、设计及开发
- 1980年获得图灵奖
- 2000年Hoare因为其在计算机科学与教育上做出的贡献被封为爵士。

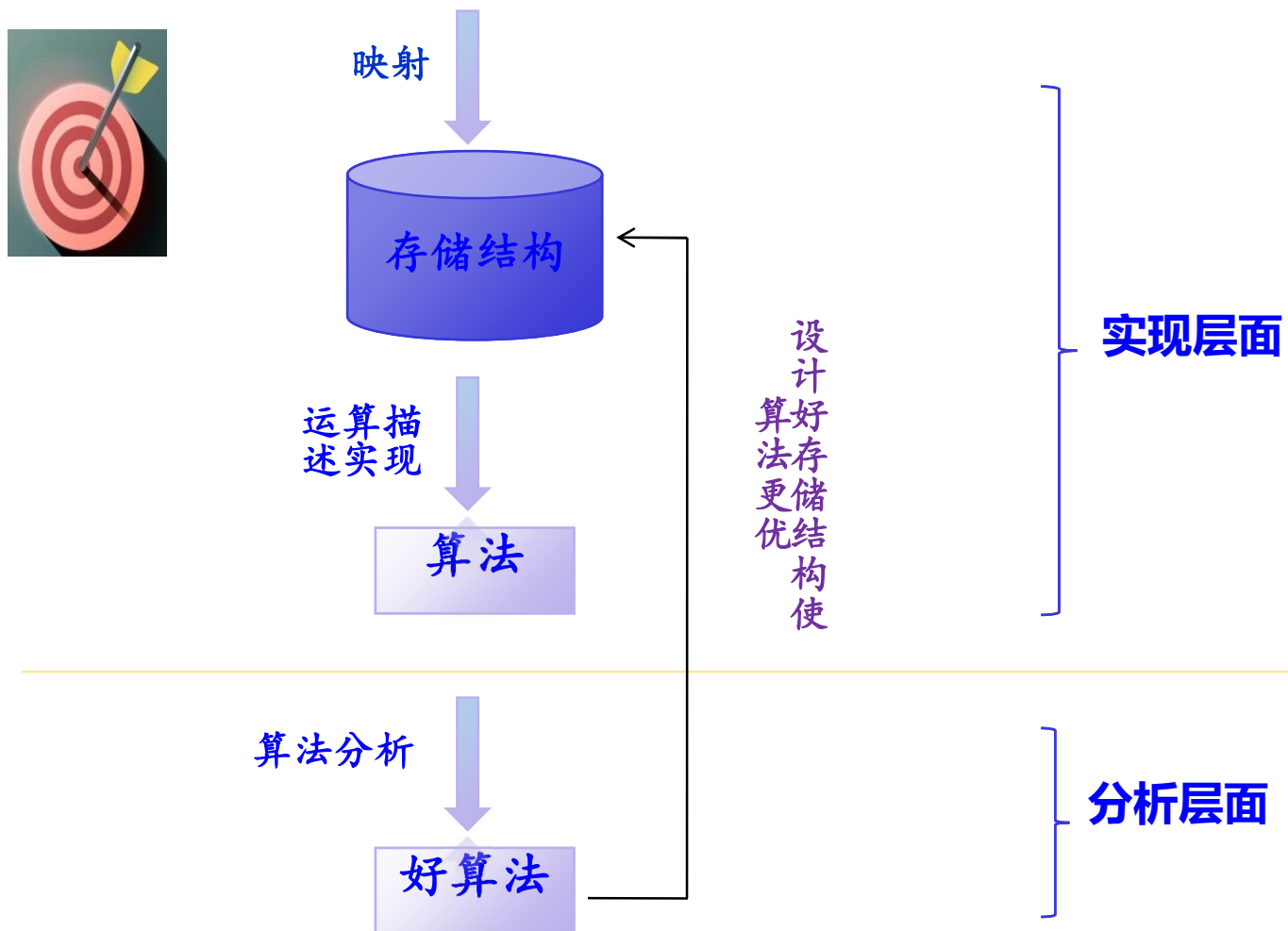
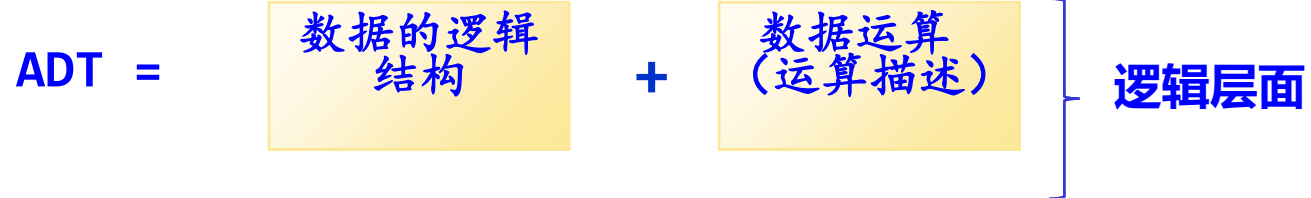


第1章 小结



从数据结构角度求解问题的过程







描述一个集合的抽象数据类型**Set**，其中所有元素为正整数，
集合的基本运算包括：

- (1) 由整数数组 $a[0..n-1]$ 创建一个集合。
- (2) 输出一个集合的所有元素。
- (3) 判断一个元素是否在一个集合中。
- (4) 求两个集合的并集。
- (5) 求两个集合的差集。
- (6) 求两个集合的交集。

在此基础上设计集合的**顺序存储结构**，并实现各基本运算的
算法。



解：抽象数据类型ASet的描述如下：

```
ADT Set
{
    数据对象:  $D = \{ d_i \mid 0 \leq i \leq n, n \text{ 为一个正整数} \}$ 
    数据关系: 无。
    基本运算:
        createset( &s, a, n): 创建一个集合s;
        dispset( s): 输出集合s;
        inset(s, e): 判断e是否在集合s中
        void add(s1, s2, s3):  $s3 = s1 \cup s2$ ; // 求集合的并集
        void sub(s1, s2, s3):  $s3 = s1 - s2$ ; // 求集合的差集
        void intersection(s1, s2, s3):  $s3 = s1 \cap s2$ ; // 求集合的交集
}
```



设计集合的顺序存储结构类型如下：

```
typedef struct          //集合结构体类型
{  int data[MaxSize];  //存放集合中的元素，其中MaxSize为常量
  int length;          //存放集合中实际元素个数
} Set;                 //将集合结构体类型用一个新类型名Set表示
```

静态分配方式



采用Set类型的变量存储一个集合。对应的基本运算算法设计如下：

```
void createset(Set &s, int a[], int n)
//创建一个集合
{   int i;
    for (i=0;i<n;i++)
        s.data[i]=a[i];
    s.length=n;
}

void dispset(Set s)                //输出一个集合
{   int i;
    for (i=0;i<s.length;i++)
        printf("%d ", s.data[i]);
    printf("\n");
}
```



```
bool inset(Set s, int e) //判断e是否在集合s中
{   int i;
    for (i=0;i<s.length;i++)
        if (s.data[i]==e)
            return true;
    return false;
}
```




```
void add(Set s1, Set s2, Set &s3)    //求集合的并集
{  int i;
    for (i=0;i<s1.length;i++) //将集合s1的所有元素复制到s3中
        s3.data[i]=s1.data[i];
    s3.length=s1.length;
    for (i=0;i<s2.length;i++) //将s2中不在s1中出现的元素复制到s3中
        if (!inset(s1, s2.data[i]))
        {  s3.data[s3.length]=s2.data[i];
            s3.length++;
        }
}
```



```
void sub(Set s1, Set s2, Set &s3)    //求集合的差集
{  int i;
   s3.length=0;
   for (i=0;i<s1.length;i++) //将s1中不出现在s2中的元素复制到s3中
       if (!inset(s2, s1.data[i]))
       {  s3.data[s3.length]=s1.data[i];
          s3.length++;
       }
}
```

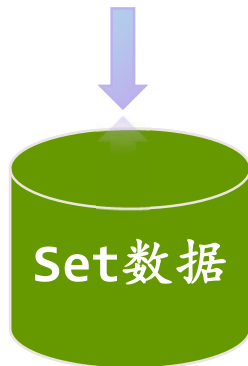


```
void intersection(Set s1, Set s2, Set &s3) //求集合的交集
{ int i;
  s3.length=0;
  for (i=0;i<s1.length;i++) //将s1中出现在s2中的元素复制到s3中
    if (inset(s2, s1.data[i]))
    { s3.data[s3.length]=s1.data[i];
      s3.length++;
    }
}
```



集合数据结构（已实现）

- `createset(Set &s, int a[], int n)`: 创建一个集合
- `dispset(Set s)`: 输出一个集合
- `inset(Set s, int e)`: 判断e是否在集合s中
- `add(Set s1, Set s2, Set &s3)`: 求集合的并集
- `sub(Set s1, Set s2, Set &s3)`: 求集合的差集
- `intersection(Set s1, Set s2, Set &s3)`: 求集合的交集





集合数据结构Set的应用

```
void main()
{
    Set s1, s2, s3, s4, s5;
    int a[]={1, 2, 3, 4, 5};
    int b[]={2, 3, 4, 5, 6, 7, 8};
    int n=5, m=7;
    createset(s1, a, n);
    createset(s2, b, m);
    printf(" s1:"); dispset(s1);
    printf(" s2:"); dispset(s2);
    printf(" s3=s1 $\cup$ s2\n");
    add(s1, s2, s3);
    printf(" s3:"); dispset(s3);
    printf(" s4=s1-s2\n");
    sub(s1, s2, s4);
    printf(" s4:"); dispset(s4);
    printf(" s5=s1 $\cap$ s2\n");
    intersection(s1, s2, s5);
    printf(" s5:"); dispset(s5);
}
```



```
"F:\清华大学数据结构\2016版\数..."
s1:1 2 3 4 5
s2:2 3 4 5 6 7 8
s3=s1 $\cup$ s2
s3:1 2 3 4 5 6 7 8
s4=s1-s2
s4:1
s5=s1 $\cap$ s2
s5:2 3 4 5
Press any key to continue.
```



2

算法描述—输出型参数

算法：输入 $\Rightarrow$ 输出



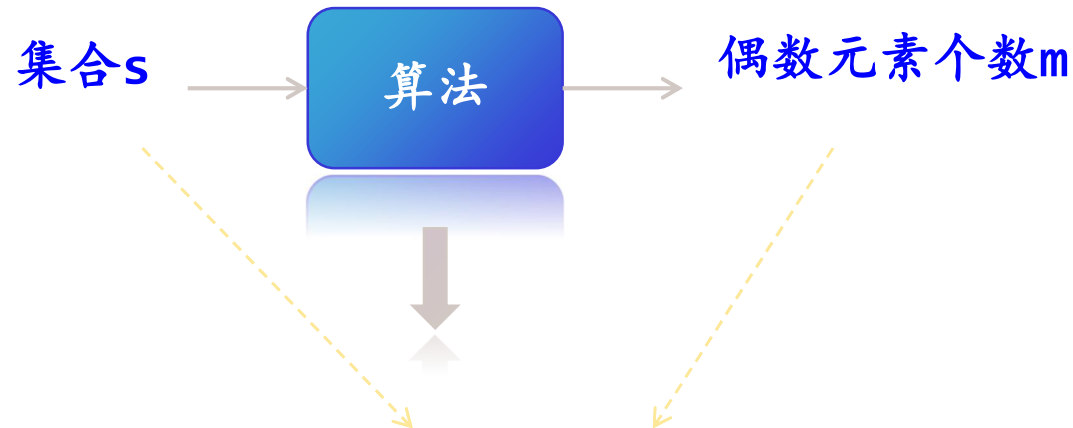
```
返回值 函数名(输入参数, 输出参数)
{
    //实现代码;
}
```



采用引用类型参数



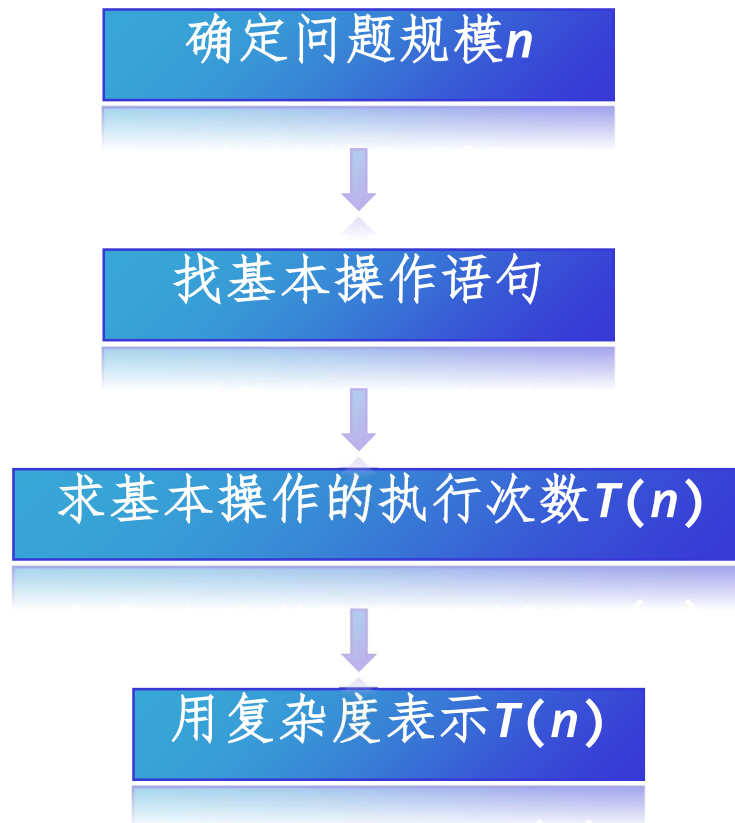
设计一个算法求整数集合s中偶数元素个数。



```
void Evennumbers(Set s, int &m)
{
    m=0;
    for (int i=0;i<s.length;i++)
        if (s.data[i]%2==0) m++;
}
```



算法时间复杂度分析





1、实践练习

- P25, 练习题I, 题号: 2、7、9
- P27, 上机实验题1, 题号: 实验题1, 实验题4
- P28, LeetCode 在线编程题1, 题号: 1

<https://leetcode.cn/problemset/>

2、提交方式:

- 要求: 要求使用C/C++
- 形式: 电子档, 需要包含题目、答案。编程类需要提供源代码和运行结果
- 提交方式: 发送到邮箱: sdzy_zqlan@163.com
- 注意: 邮件主题及文件名需要备注好姓名、学号、第一次实践作业